CASE STUDY

Spend Control Console

Building an AI-assisted enterprise spend governance platform with split services, cloud-ready infrastructure, and production-structured delivery.

FRONTEND	BACKEND	AI LAYER
Next.js, TypeScript, Tailwind	FastAPI microservices, PostgreSQL	Ollama with deterministic guardrails

Executive Summary

Spend Control Console is a business-centric expense governance platform that helps companies control employee spending, enforce policy, detect anomalies, and support finance teams with AI-assisted explanations and summaries. The system was designed as a production-structured platform rather than a single-app prototype, with separate frontend and service repositories, formal deployment assets, and Azure-ready infrastructure.

The platform supports employee claim submission, manager approvals, finance review workflows, deterministic policy checks, anomaly detection, receipt handling groundwork, AI summaries, and multi-environment deployment through local development, Docker Compose, Kubernetes manifests, and Azure infrastructure.

Business Problem

Many expense tools are either too simplistic for enterprise governance or too heavyweight to customize and operate easily. The goal of this project was to create a modern internal spend control platform that balances usability, finance policy enforcement, anomaly visibility, and operational extensibility.

The product needed to solve for three groups at once:

- Employees needed a fast and structured way to submit expenses and upload receipts.
- Managers needed approval visibility with enough context to make quick decisions.
- Finance teams needed stronger policy control, dashboarding, auditability, and explainable anomaly review.

Project Goals

- Build a real enterprise expense platform instead of a UI-only demo.
- Keep service boundaries clean enough for future independent ownership.
- Use AI as an assistive layer, not as the final policy decision-maker.
- Support local, Docker Compose, Kubernetes, and Azure deployment paths.
- Design infrastructure with a realistic production mindset from the beginning.
- Create an internal UI that feels premium and operational rather than consumer-fintech themed.

Solution Overview

The platform was built around a three-tier architecture:

- Presentation layer: Next.js frontend.
- Service layer: control-service, expense-service, and ai-service.
- Data/model layer: PostgreSQL and Ollama.

The frontend primarily talks to `control-service`, which serves as the public API surface and orchestration layer. The control service calls the expense and AI services internally, which keeps the browser-facing contract simpler and more stable.

Architecture Design

Frontend

- Next.js App Router
- TypeScript
- Tailwind CSS
- Dashboard-oriented navigation and page structure

Service Layer

- **control-service**: auth, RBAC, policy, approvals, summaries, orchestration
- **expense-service**: claims, receipts, budgets, vendors, audit logs
- **ai-service**: explanations, summaries, chat assistant, Ollama integration

Data and AI

- PostgreSQL with SQLAlchemy and Alembic
- Ollama for local model serving
- Pydantic-based settings and contracts

Developer Platform

- Split repositories with a dedicated platform repo
- Docker Compose layering
- Kubernetes manifests
- Terraform for Azure AKS and Azure VM paths

Functional Scope

- Login and role-based access for employee, manager, and finance admin roles.
- Expense submission with receipt upload.
- Claim tracking and approval flows.
- Department and category tagging.
- Budget and dashboard views.
- Anomaly inbox for finance review.
- Audit trail visibility.
- AI-powered explanations, summaries, and chat assistance.

Policy and Anomaly Strategy

A central design rule was that AI must never make final policy decisions. Deterministic business rules remain the source of truth, while AI is used to explain, summarize, categorize, and assist.

Examples of deterministic logic implemented in the platform include:

- Receipt required above a configurable threshold.
- Category-based spend caps.
- Weekend restrictions for selected categories.
- Duplicate claim detection.
- Rounded-amount suspicion flags.
- Frequent high-value same-vendor anomaly flags.
- Department over-budget warnings.

Data Model

The project uses a single PostgreSQL instance for simplicity, but service ownership was kept clean so it can evolve into a more distributed model later. Core entities include companies, departments, users, roles, expense claims, receipts, vendors, budgets, policy rules, approval steps, anomalies, audit events, and AI conversation history.

> The design balances near-term delivery simplicity with longer-term service separation. One database is used operationally, but table ownership and service boundaries were planned with future extraction in mind.

Frontend Experience

The UI was designed as a premium B2B internal operations console. The visual direction emphasizes clarity, governance, and business structure rather than playful consumer-finance styling.

- Dashboard-oriented layout with left navigation and top context controls.
- Dedicated pages for login, claims, approvals, anomalies, finance views, budgets, and policies.
- Clear states for loading, empty, success, and failure conditions.
- Structured layouts that support both individual employees and finance operators.

Repository and Delivery Evolution

The project began as a monorepo, but was later split into separately managed repositories:

- spend-control-platform
- spend-control-frontend
- spend-control-control-service
- spend-control-expense-service
- spend-control-ai-service

This transition required making each codebase more self-contained, adapting Dockerfiles, standardizing environment handling, and moving infrastructure concerns into the platform repo. The split improved future maintainability and aligned the system more closely with real team ownership patterns.

Deployment Strategy

Local Development

Each service and the frontend can run independently for direct debugging and faster iteration.

Docker Compose

Compose was split into three layers to make operational intent clearer:

- Infrastructure compose: PostgreSQL and Ollama.
- Backend compose: control-service, expense-service, ai-service.
- Frontend compose: Next.js app.

Kubernetes

Plain YAML manifests were created for namespace, config, secrets template, deployments, services, ingress, PVCs, and migration workflows.

Azure Infrastructure Design

Two Terraform infrastructure paths were designed:

- **AKS path** for longer-term Kubernetes-oriented operations.
- **VM-Docker path** for simpler Azure deployment using regular Linux VMs.

The VM-Docker path uses one resource group, one VNet, separate subnets for frontend, backend, and data/AI tiers, and Azure Application Gateway WAF v2 at the edge.

At a high level the VM topology is:

- One frontend VM
- One backend VM
- One data/AI VM for PostgreSQL and Ollama

Major Engineering Challenges

Cloud Networking Assumptions

A critical issue surfaced when Docker-local hostnames such as `postgres` and `ollama` were still being used in Azure VM mode. Those names work in local Docker networking, but they do not resolve across separate Azure VMs. The backend services had to be updated to support full environment overrides so they could use the data VM private IP instead.

Terraform App Gateway Association

A plan-time Terraform issue appeared because a resource count depended on apply-time values. This was fixed by introducing an explicit boolean control for Application Gateway backend association, which made the plan deterministic.

Azure WAF Configuration Changes

Azure deprecated direct inline WAF configuration in Application Gateway. The implementation was updated to use a standalone WAF policy associated with the gateway.

VM SKU and Region Compatibility

Some initially chosen VM sizes were invalid for the selected Azure region, which required reworking the defaults to use accepted SKUs.

Bootstrap Reliability

An attempt to extend cloud-init to bootstrap frontend and backend app stacks made the data VM bootstrap unstable. The safe resolution was to restore the simpler and previously working Docker plus Postgres plus Ollama bootstrap for the data VM, rather than keep pushing excessive orchestration into cloud-init.

Quality and Verification Work

The delivery included repeated validation across code, infrastructure, and packaging layers. This included:

- Terraform validation and formatting.
- Docker Compose config validation.
- Backend test execution with pytest.
- Ruff checks for Python repos.
- Frontend linting and production build verification.
- Smoke tests and stale configuration scans.

This validation work mattered because the hardest failures were not syntax problems. They were integration and deployment mismatches between local development assumptions and cloud runtime behavior.

Key Decisions That Shaped the Outcome

- AI was kept assistive rather than authoritative.
- Public browser traffic was kept focused through the control service.
- Local Docker mode and Azure VM mode were treated as distinct operational environments.
- Service separation was preserved even while using one PostgreSQL instance initially.
- Infrastructure was documented and implemented as a first-class concern rather than an afterthought.

Outcome

The result is a realistic enterprise spend platform foundation: modular, cloud-aware, deployment-ready, and structured for future evolution. It provides a strong base for budget enforcement, approval workflows, anomaly review, auditability, and AI-enhanced finance operations.

Just as importantly, it captures the kinds of real implementation and infrastructure issues that matter in production projects: service discovery boundaries, bootstrap reliability, deployment mode differences, documentation drift, and environment-specific behavior.

Future Roadmap

- Move secrets into Azure Key Vault or a comparable secret store.
- Add private DNS for VM-to-VM service addressing.
- Use stronger VM startup orchestration for app services.
- Improve App Gateway diagnostics and backend probe visibility.
- Add deeper CI/CD automation for infra validation and deployment.
- Decide whether VM mode remains primary or becomes a stepping stone to AKS.

Spend Control Console case study generated from the project workspace and delivery history. Prepared as a detailed engineering and architecture narrative.